

WHATXIA · MOBILITY

WhatXia Basic

Biblia Técnica MVP v1.0

Baseline para pruebas reales

Proyecto	WhatXia Basic
Documento	Biblia Técnica MVP v1.0
Versión	1.0.0
Commit de referencia	d8a65d7
Base de datos	Supabase (29 migraciones)
Fecha	24 de julio de 2026
Estado	Baseline para pruebas reales
Clasificación	Respaldo técnico oficial · Uso interno

Consolidado desde  · Repositorio WhatXia/whatxia-mobility-mvp · Canal WhatsApp Cloud API · Stack Next.js + Supabase + Vercel

Tabla de contenido

01 Resumen ejecutivo

02 Estado del MVP

03 Arquitectura

04 Repositorio (GitHub)

05 Vercel

06 Supabase

07 Integración WhatsApp / Meta

08 Conocimiento funcional

09 Reglas de negocio y decisiones de arquitectura

10 Árbol del proyecto

11 Manual de recuperación

12 Próximos pasos

La numeración de páginas aparece en el pie de cada hoja. Los enlaces de este índice corresponden a los capítulos del documento.

1. Resumen ejecutivo

WhatXia Basic (repositorio `whatxia-mobility-mvp`) es el MVP de movilidad urbana por WhatsApp enfocado en **Ibagué**. Este documento consolida el backup integral ubicado en `docs/backup/` y constituye la **Biblia Técnica MVP v1.0**: referencia oficial para archivar, compartir y recuperar el sistema.

Campo	Valor
Proyecto	WhatXia Basic
Documento	Biblia Técnica MVP v1.0
Versión	1.0.0
Fecha	24 de julio de 2026
Commit de referencia	<code>d8a65d7</code> (<code>d8a65d7513906117aef52e89d87edd8e344a69ad</code>)
Rama	<code>main</code>
Repositorio	<code>https://github.com/WhatXia/whatxia-mobility-mvp</code>
Base de datos	Supabase (29 migraciones, 001–029)
Hosting	Vercel (Next.js 16)
Canal	WhatsApp Cloud API (Meta)
Estado	Baseline para pruebas reales

Alcance de este backup

- Auditoría y documentación únicamente (sin alterar código de producto ni ejecutar migraciones en este sprint documental).
- Secretos: solo se documentan *nombres* de variables de entorno, nunca valores.
- Fuentes: `docs/backup/01 ... 07`, `arbol-proyecto.txt` y metadatos Git del corte `d8a65d7`.

Contenido consolidado

1. Estado del MVP
2. Arquitectura y conocimiento funcional
3. Repositorio, Vercel y Supabase (incl. 29 migraciones)
4. Integración WhatsApp / Meta
5. Reglas de negocio
6. Árbol del proyecto
7. Manual de recuperación
8. Próximos pasos

2. Estado del MVP

Corte: 24 de julio de 2026 · `main` @ `d8a65d7` · migraciones hasta **029**.

Qué funciona

Capacidad	Notas
Webhook WhatsApp verify + firma + parse	<code>/api/webhook</code>
Menú pasajero / menú conductor	Mismo número WABA
Solicitud de servicio (menú + texto libre + voz si hay key)	Mobility booking
Origen (pin/texto) y destino (Places + confirmación)	Geo Google
Cotización estimada con rango +\$3.000	Sin recargo \$800 en el estimado
Creación de trip + búsqueda + oferta	Dispatch
Aceptar / rechazar / ETA / llegar / iniciar / navegar / finalizar	Ciclo conductor
Mensajes de fin con rango estimado + copy taxímetro/\$800	Usa <code>quoted_fare</code>
Reasignación / exclusiones / waiting flow	Search timeouts
Cancelaciones + políticas	Pasajero y conductor
Túnel P↔D + cierre programado	Cron tunnels
Rating post-viaje	Pasajero
Registro conductor (welcome Continuar/Abandonar, cancel/salir/resume)	Formulario nuevo 029
Actualización de datos / docs vencidos + cron documentos	Drivers
Tariff Engine SSoT (<code>fare_rules</code> + <code>holidays</code>)	Ibagué v2
Taxímetro de prueba (calibración, sin trips)	Tablas 025–027
City context Ibagué	<code>cities</code>

Qué está pendiente

Ítem	Detalle
Panel admin / BI	No existe UI operativa
Multi-ciudad en runtime	Modelo listo; ops = Ibagué
Analytics / rendimiento conductor	Stubs en menú ("Pronto...")
Reportes conductor	Stub
README de producto	Sigue siendo boilerplate Next
Landing web de producto	No es el canal real

Ítem	Detalle
RLS / Auth Supabase	No implementados en migraciones
Captura taxímetro físico en calibración	Opcional/null en MVP
Cron search en <code>vercel.json</code>	Solo documents + tunnels; search también vía webhook
Términos y Condiciones / tratamiento de datos en registro	Explícitamente aplazado
Flujo de aprobación conductor (ops humana)	Mensaje de “validación del equipo”; automatización de aprobación pendiente de producto
Calibración estadística tarifa en calle	Taxímetro listo; falta muestra N corridas
Dominios/secrets de prod en este backup	Confirmar en dashboards (no versionados)

Bugs / fricciones conocidas o de diseño

Tema	Descripción
Handler responde 200 ante error interno	Evita reintentos agresivos de Meta; puede “tragarse” fallos → vigilar logs
Destinos ambiguos Places	Recuperación UX existe; aún hay riesgo de mal match
Dual rol mismo número	Edge cases si un conductor pide viaje como pasajero en paralelo
 vs taxímetro	Intención bare abre módulo conductor; no confundir con inicio de taxímetro
Informe previo desactualizado	<code>docs/INFORME-ESTADO-...</code> corta en 027; este backup es la referencia a 029
Sin RLS	Riesgo si se filtra anon key o se abre API client-side
Dependencia webhooks	Caídas Meta/Vercel = bot caído

Riesgos

- 1. **Tarifa / confianza de negocio** — estimado vs taxímetro real aún en calibración.
- 2. **Secretos** — service role + WhatsApp token en Vercel; sin RLS.
- 3. **Geo** — calidad de Places/Routes en Ibagué.
- 4. **Ops manual** — aprobación de conductores y soporte sin panel.
- 5. **Ventana 24h WhatsApp** — mensajes proactivos limitados sin plantillas.
- 6. **Conocimiento en chat** — mitigado por este backup; mantener docs al día.

Madurez (estimación)

Dimensión	Nivel
Piloto controlado 1 ciudad	~75–85%
Expansión multi-ciudad + panel	~40%

Dimensión	Nivel
Cumplimiento/docs legales en flujo	Bajo (pendiente T&C)

3. Arquitectura

El **Core Agent** es el orquestador único de entrada (`handleIncomingMessage`). No hay microservicios: un deploy Next.js en Vercel recibe el webhook y enruta por prioridad hacia Mobility, Dispatch, registro de conductores, túneles, taxímetro de prueba, etc.

```
Meta Webhook
  → verify + parse + normalize (voz)
  → Core Agent (handler)
    → tunnels / rating / cancellations / search
    → dispatch / driver menu / registration
    → taximeter test / booking (Mobility)
    → greeting / intent / fallback
  → WhatsApp client → usuario
  → Supabase · Google Maps · OpenAI (opcional)
```

Conversation Planner (concepto): máquina de estados en `conversation_sessions` + orden de evaluación del handler. **Response Generator** (concepto): plantillas determinísticas vía cliente WhatsApp (no LLM de negocio; Whisper solo convierte audio→texto).

Arquitectura del Core Agent

Nombre de producto: Core Agent

Implementación: `src/lib/whatsapp/handler.ts` → `handleIncomingMessage`

Es el orquestador único de entrada. No hay microservicios: un proceso Next.js en Vercel recibe el webhook y enruta por prioridad.

Capas

```
Meta Webhook
  → verify + parse + normalize (voz)
  → Core Agent (handler)
    → tunnels / rating / cancellations / search timeouts
    → dispatch buttons
    → driver menu / registration / update / docs
    → taximeter test
    → booking (Mobility)
    → greeting / intent / fallback
  → WhatsApp client (salida)
  → Supabase (persistencia)
  → Google Maps / OpenAI (servicios externos)
```

Conversation Planner (concepto → código)

No existe un paquete llamado `conversation-planner`. La “planificación” es la **máquina de estados** en:

Persistencia	Código
<code>conversation_sessions.state</code> + drafts	<code>src/lib/sessions.ts</code>
Estados de booking	<code>src/types</code> + <code>src/lib/booking/flow.ts</code>

Persistencia	Código
Estados registro conductor	<code>DRIVER_REGISTERING</code> , <code>DRIVER_REGISTRATION_WELCOME</code> , <code>PAUSED</code> , <code>RESUME_CHOICE</code> , ...
Taxímetro	<code>taximeter_test_sessions.state</code>

El planner efectivo = **orden de if en el handler** + estado en Supabase.

Response Generator (concepto → código)

No hay generador LLM de respuestas de negocio. Las respuestas son **plantillas determinísticas**:

- `sendMessage` / `sendButtonsMessage` (`src/lib/whatsapp/client.ts`)
- Copy embebido en booking, dispatch, registration, rating, etc.

Excepción: **voz entrante** usa Whisper (LLM/ASR) solo para convertir audio→texto; el texto entra al mismo pipeline.

Mobility (Booking)

Código: `src/lib/booking/flow.ts`, `src/lib/booking/intent.ts`

Flujo pasajero (happy path)

1. Saludo / menú • texto con intención de servicio (`parseMobilityIntent`).
2. Origen: pregunta “¿Dónde te recogemos?” → pin WhatsApp y/o texto.
3. Destino: texto → Places (bias ciudad) → confirmación / recuperación.
4. Cotización: `estimateFare` → presentación de **rango** (`present-estimate.ts`, margen +\$3.000).
5. Confirmación → crea `trip` `SEARCHING` → dispatch.

Reglas UX relevantes

- No auto-completar “Punto de recogida” sin interacción.
- Intención libre evita depender siempre de “Hola”.
- Mensaje de fin de viaje usa el **mismo** `quoted_fare` del inicio (rango), no recalcula para mostrar.

Dispatch

Código: `src/lib/dispatch.ts` (+ `search.ts`, `trip-exclusions.ts`, `waiting-flow.ts`)

Ciclo conductor

Oferta → Aceptar/Rechazar → ETA → Ver ubicación / Llegué → Iniciar → Navegar destino → Finalizar.

Comportamientos

- Oferta incluye ubicación nativa WhatsApp + rango estimado.
- Exclusiones tras cancelación/rechazo para reasignar.

- Timeouts de búsqueda + “continuar / cancelar” (waiting flow).
- Al finalizar: persiste `final_fare` vía `finalizeFare`, notifica P/D con rango estimado + copy de taxímetro/\$800, abre rating, programa cierre de túnel.

Pricing / Tariff Engine

Código: `src/lib/tariff/*` (entrada `src/lib/tariff/index.ts`)

Legacy: `src/lib/pricing/*` (deprecado; no SSoT)

SSoT

Fuente	Tabla / artefacto
Parámetros	<code>public.fare_rules</code>
Festivos	<code>public.holidays</code>
Ciudad	<code>public.cities</code> + <code>src/lib/city/context.ts</code>

API

- `estimateFare` — cotización informativa pre-aceptación.
- `finalizeFare` — tarifa oficial al terminar viaje.
- Presentación: `formatEstimatedFareRange*` / `formatCopSymbol`.

Reglas de negocio tarifarias (Ibagué v2 — resumen)

- Base / piso: tarifa mínima (p. ej. \$6.600).
- Distancia: incrementos sobre excedente tras ~1.600 m; tick 80 m × \$90 (028).
- Recargos configurables: nocturno 19:00–06:00, domingo/festivo, aeropuerto, plataforma.
- Flag `APPLY_CALL_SURCHARGE_ON_ESTIMATE = false`: el recargo de solicitud **no** se suma al estimado mostrado; el copy de fin de viaje recuerda **+\$800** sobre taxímetro.
- UI de estimado: rango `[X, X+3000]` redondeado a centenas.

Decisión de arquitectura

El motor **no** usa `date-holidays` en runtime; solo lee `holidays` en Supabase. Los archivos `city-config/*.ts` son referencia/seed históricos, no fuente operativa.

Mapa rápido archivo → dominio

Dominio	Path
Core Agent	<code>src/lib/whatsapp/handler.ts</code>
WhatsApp I/O	<code>src/lib/whatsapp/*</code> , <code>src/app/api/webhook</code>
Mobility	<code>src/lib/booking/*</code>
Dispatch	<code>src/lib/dispatch.ts</code>

Dominio	Path
Search / waiting	<code>src/lib/search.ts</code> , <code>waiting-flow.ts</code>
Tariff	<code>src/lib/tariff/*</code>
Geo	<code>src/lib/geo/*</code>
Drivers	<code>driver-registration</code> , <code>driver-menu</code> , <code>driver-update</code> , <code>supabase/drivers</code>
Tunnels	<code>src/lib/tunnels.ts</code>
Cancellations	<code>src/lib/cancellations.ts</code>
Rating	<code>src/lib/rating.ts</code>
Voice	<code>src/lib/voice/*</code>
Taximeter	<code>src/lib/taximeter-test/*</code>
City	<code>src/lib/city/context.ts</code>
Crons	<code>src/app/api/cron/*</code>

4. Repositorio (GitHub)

Identidad

Campo	Valor
Organización / repo	WhatXia/whatxia-mobility-mvp
URL	https://github.com/WhatXia/whatxia-mobility-mvp
Remote origin	https://github.com/WhatXia/whatxia-mobility-mvp.git
Rama principal	main
HEAD documentado	d8a65d7513906117aef52e89d87edd8e344a69ad
Mensaje HEAD	refactor(mobility): align trip completion fare messaging
Nombre npm	whatxia-mobility-mvp
Versión package	0.1.0
Privacidad	Verificar en GitHub (CLI <code>gh</code> no disponible en el entorno de auditoría)

Ramas relevantes

Rama	Notas
main	Rama de trabajo y despliegue esperada
backup/mobility-mvp-v0.1	Backup histórico; en el entorno local aparecía <i>ahead 1</i> respecto a su remote

Estado del código (corte del backup)

- Working tree alineado con `main ... origin/main` en el momento de la auditoría (sin divergencia reportada por `git status -sb`).
- Stack: **Next.js 16.2.10**, **React 19.2.4**, **TypeScript 5**, **Supabase JS 2.x**.
- Producto real = bot WhatsApp (`src/lib/**` + `src/app/api/webhook`). La página web (`src/app/page.tsx`) es landing starter de Next, no el producto de negocio.

Dependencias (`package.json`)

Runtime

Paquete	Rol
next	Framework / App Router / API routes
react / react-dom	UI (mínima; producto es API + WhatsApp)
@supabase/supabase-js	Cliente Supabase (service role en servidor)

Dev

Paquete	Rol
<code>typescript</code>	Tipado
<code>eslint</code> / <code>eslint-config-next</code>	Lint
<code>@types/*</code>	Tipos
<code>date-holidays</code>	Solo seed de festivos (script); no runtime del Tariff Engine

Scripts npm

Script	Comando	Uso
<code>dev</code>	<code>next dev</code>	Desarrollo local
<code>build</code>	<code>next build</code>	Build producción (Vercel)
<code>start</code>	<code>next start</code>	Servir build
<code>lint</code>	<code>eslint</code>	Lint

No hay scripts `test` / `certify` en `package.json`. Existen archivos `*.certify.ts` ejecutables vía `npx tsx` / Node según convención del equipo.

Scripts auxiliares (`scripts/`)

Archivo	Propósito
<code>diagnose-dispatch-schema.mts</code>	Diagnóstico de esquema de despacho
<code>generate-co-holidays-sql.ts</code>	Genera SQL seed de festivos CO (alimentó migración 024)
<code>places-ibague-diag.ts</code>	Diagnóstico Places Ibague
<code>wa-offer-location-probe.ts</code>	Sondeo de ofertas/ubicación WhatsApp

Estructura de alto nivel

```
whatxia-mobility-mvp/
├─ docs/                # Documentación (incluye este backup)
├─ public/              # Assets estáticos Next
├─ scripts/             # Utilidades ops / diagnóstico
├─ src/
│  ├─ app/
│  │  ├─ api/
│  │  │  ├─ webhook/    # Meta WhatsApp webhook
│  │  │  └─ cron/       # documents, tunnels, search
│  │  └─ layout.tsx
│  └─ page.tsx          # Landing starter (no producto)
├─ lib/                 # Dominio: booking, dispatch, tariff, drivers, ...
├─ types/
├─ supabase/
│  ├─ migrations/       # 001 ... 029
│  └─ APPLY_SPRINT_18.sql
├─ .env.example
├─ package.json
├─ vercel.json          # Crons Vercel
├─ next.config.ts
├─ tsconfig.json
├─ AGENTS.md / CLAUDE.md # Notas para agentes (Next breaking changes)
└─ README.md            # Aún boilerplate create-next-app
```

Árbol detallado de archivos: [arbol-proyecto.txt](#).

Commits relevantes (historial reciente de `main`)

Orden: más reciente primero (corte del backup).

Commit	Tema
d8a65d7	Mensajes de fin de viaje: rango estimado (no tarifa final)
45f9628	Formulario registro conductor + schema email/tarjeta operación
51d57c7	Inicio registro: Continuar / Abandonar
57e0e94 / 6186a31	Mensaje final registro + sin confirmaciones intermedias
2dcda30	Cancelar inscripción / Salir / reanudar
f62c7e1	Intent conductor 🚗 / 🚕 → menú o registro
11d40ef / 91f291d / 4c44fa5	Rango de tarifa estimada (pasajero/conductor)
a355aee	Flag: sin recargo de solicitud en estimado
b5f104e / 028	Tarifa Ibagué v2 (mínimo 1600 m, tick \$90)
c594367 ... cb5dee5	Taxímetro de prueba
0371750	Voz WhatsApp → Whisper E2E
e8f8416 / a2e67f7	Intención de viaje en texto libre
5117bd9 ... b9893d7	Tariff SSoT + nocturno + holidays

Commit	Tema
1d8f7a1 ...	Destination recovery / Places UX
Sprints 13–21 (históricos)	Trips, passengers, tunnels, cancelaciones, search

Para listado completo: `git log --oneline`.

Variables de entorno documentadas en repo

Fuente canónica de **nombres**: `.env.example` (+ extras vistos en entorno local de desarrollo).

Ver [02-vercel.md](#) y [07-manual-recuperacion.md](#).

Notas para agentes / Next

- `AGENTS.md` / `CLAUDE.md`: esta versión de Next puede diferir del entrenamiento; consultar docs en `node_modules/next/dist/docs/` antes de cambios de API.

5. Vercel

Proyecto conectado

Campo	Valor documentado
Repo Git	WhatXia/whatxia-mobility-mvp
Framework	Next.js (App Router)
Directorio raíz	/ (monorepo no aplica)
Rama de producción esperada	main
CLI Vercel en entorno de auditoría	No disponible (vercel CLI ausente)

Los nombres exactos del proyecto en el dashboard, team ID y dominios custom **deben confirmarse en <https://vercel.com>** (Project Settings). Este backup documenta lo inferible desde el repositorio y la configuración versionada.

Build y runtime

Setting	Valor
Install	npm install (default Vercel)
Build Command	next build (package.json → build)
Output	Next.js (Vercel adapter automático)
Node	Compatible con Next 16 (entorno local auditado: Node v24.18.0 ; en Vercel usar LTS soportada por Next 16)
next.config.ts	Config vacía (defaults)

Crons (vercel.json)

```
{
  "crons": [
    { "path": "/api/cron/documents", "schedule": "0 13 * * *" },
    { "path": "/api/cron/tunnels", "schedule": "5 13 * * *" }
  ]
}
```

Path	Schedule (UTC)	Propósito
/api/cron/documents	0 13 * * *	Recordatorios / bloqueo docs conductores
/api/cron/tunnels	5 13 * * *	Cierre de túneles conversacionales vencidos

También existe `/api/cron/search` (timeouts de búsqueda). **No** está registrado en `vercel.json`; el timeout también se dispara al recibir webhooks vía `processDueSearchTimeouts()`.

Los crons validan `CRON_SECRET` (header/query según implementación de cada route).

Dominios

Tipo	Estado en backup
Dominio <code>*.vercel.app</code>	Asignado por Vercel al proyecto (confirmar en Dashboard → Domains)
Dominio custom	No versionado en repo; confirmar en Dashboard
Webhook Meta	Debe apuntar a <code>https://<dominio-produccion>/api/webhook</code>

Variables de entorno (solo nombres)

Documentadas en `.env.example`

Nombre	Uso
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL proyecto Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Anon key (presente en example; el runtime usa service role)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Acceso servidor (bypass RLS)
<code>WHATSAPP_TOKEN</code>	Token Cloud API
<code>WHATSAPP_PHONE_NUMBER_ID</code>	Phone Number ID
<code>WHATSAPP_VERIFY_TOKEN</code>	Verificación webhook GET
<code>WHATSAPP_APP_SECRET</code>	Firma HMAC <code>X-Hub-Signature-256</code>
<code>WHATSAPP_API_VERSION</code>	Default <code>v21.0</code>
<code>OPENAI_API_KEY</code>	Whisper (voz)
<code>OPENAI_WHISPER_MODEL</code>	Default <code>whisper-1</code>
<code>OPENAI_WHISPER_LANGUAGE</code>	Default <code>es</code>
<code>VOICE_TRANSCRIPTION_PROVIDER</code>	Default <code>openai_whisper</code>
<code>CRON_SECRET</code>	Autenticación de crons
<code>GOOGLE_MAPS_API_KEY</code>	Places / Geocoding / Routes
<code>GEO_CITY_LAT</code>	Fallback bias ciudad
<code>GEO_CITY_LNG</code>	Fallback bias ciudad
<code>GEO_CITY_RADIUS_M</code>	Fallback radio
<code>PLACE_CONFIDENCE_THRESHOLD</code>	Umbral Places

Nombres observados en `.env.local` local (sin valores)

Además de un subconjunto de las anteriores:

Nombre	Notas
<code>MOBILITY_ROUTE_PROVIDER</code>	Presente en local; no está en <code>.env.example</code> — confirmar si se usa en prod

Variables de plataforma Vercel (inyectadas)

Nombre	Uso
<code>VERCEL</code>	Detectado en <code>geo/config</code>
<code>VERCEL_ENV</code>	<code>production</code> / <code>preview</code> / <code>development</code>
<code>NODE_ENV</code>	Runtime Node

Nunca versionar valores. En Vercel: Project → Settings → Environment Variables (Production / Preview / Development según corresponda).

Estado actual (según repo)

- Deploy esperado: cada push a `main` (si está conectado el Git Integration).
- Funcionalidad productiva: webhook WhatsApp + crons documentos/túneles.
- Landing web no es el producto; health del sistema = logs de `/api/webhook` y Supabase.

Checklist ops Vercel

1. Proyecto vinculado al repo GitHub correcto.
2. Todas las variables de la tabla (excepto opcionales de voz/geo fallback) configuradas en **Production**.
3. Crons activos (plan Vercel que soporte Cron Jobs).
4. Dominio de producción pegado en Meta webhook.
5. Redeploy tras cambiar env vars.

6. Supabase

Rol en la arquitectura

Supabase es la **fuentes de verdad de dominio** (conductores, viajes, sesiones, tarifas, túneles, taxímetro de prueba).

El backend Next.js se conecta con:

- `NEXT_PUBLIC_SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY` (cliente en `src/lib/supabase/client.ts`)

El código de servidor usa **service role**. No hay flujo de login Supabase Auth para pasajeros/conductores: la identidad es el **teléfono de WhatsApp**.

Auth

Tema	Estado
Auth de usuarios finales (email/OAuth)	No usado por el MVP WhatsApp
Identidad	Teléfono WhatsApp normalizado
API keys	Service role en servidor; anon key documentada pero no es el path principal del bot
RLS	No hay políticas RLS creadas en las 29 migraciones

Implicación de seguridad: las tablas dependen de que las keys no se filtren. El acceso público vía anon sin RLS sería riesgoso; el diseño asume acceso solo desde el backend con service role.

Storage / Buckets

Tema	Estado
Buckets Storage	Ninguno definido en migraciones
Media WhatsApp	Se descarga vía Graph API (<code>WHATSAPP_TOKEN</code>); no se persiste en Supabase Storage en este MVP

Funciones SQL / Triggers

Tema	Estado en migraciones
<code>CREATE FUNCTION</code>	No hay funciones PL/pgSQL de negocio versionadas
Triggers	No hay triggers versionados
Defaults / constraints	Sí: PK, FK, CHECK de status, índices

La lógica de negocio vive en TypeScript (`src/lib/**`), no en stored procedures.

Tablas (inventario)

Actores

Tabla	Propósito
<code>drivers</code>	Conductores: perfil, vehículo, docs, disponibilidad, status, ciudad
<code>passengers</code>	Pasajeros (alta automática por teléfono)

Viajes y operación

Tabla	Propósito
<code>trips</code>	Ciclo de vida del servicio + geo + tarifas
<code>trip_cancellations</code>	Historial de cancelaciones / causales
<code>trip_driver_exclusions</code>	Conductores excluidos de un trip (reasignación)

Conversación

Tabla	Propósito
<code>conversation_sessions</code>	Estado FSM por teléfono + drafts
<code>conversation_tunnels</code>	Túnel P↔D ligado a trip
<code>tunnel_messages</code>	Mensajes del túnel

Documentos

Tabla	Propósito
<code>document_reminders</code>	Recordatorios de vencimiento enviados

Ciudad / tarifa

Tabla	Propósito
<code>cities</code>	Contexto de ciudad (activa: Ibagué)
<code>fare_rules</code>	SSoT parámetros tarifarios por ciudad
<code>holidays</code>	Festivos nacionales (CO) para recargo

Calibración

Tabla	Propósito
<code>taximeter_test_sessions</code>	Sesión efímera de prueba (PK teléfono)
<code>taximeter_test_runs</code>	Corridas de calibración (sin FK a <code>trips</code>)

Relaciones principales

```
cities 1—* fare_rules
cities 1—* drivers
cities 1—* trips

passengers 1—* trips
drivers 1—* trips (assigned)
trips 1—* trip_cancellations
trips 1—* trip_driver_exclusions
trips 1—* conversation_tunnels
conversation_tunnels 1—* tunnel_messages
drivers 1—* document_reminders
drivers 1—* taximeter_test_runs (nullable)
drivers 1—* taximeter_test_sessions (nullable)
```

- Taxímetro **no** referencia `trips` (independiente de Mobility).
- `conversation_sessions.phone` es PK (estado conversacional por número).

Columnas clave recientes en `drivers` (029)

Además del perfil histórico (docs, vehículo, emergencia):

- `email`
- `operation_expires_at` (tarjeta de operación)

Campos de emergencia / `vehicle_year` siguen en esquema (nullable) pero **fuera** del flujo de registro actual.

Políticas RLS

Ninguna política `CREATE POLICY` / `ENABLE ROW LEVEL SECURITY` aparece en `supabase/migrations/*`.

Documentar como decisión actual del MVP: seguridad perimetral vía service role + secretos de Vercel, no RLS por fila.

Migraciones 001–029

Aplicar **en orden numérico**. Entorno productivo debe estar al día hasta **029**.

#	Archivo	Qué hace
001	<code>001_create_drivers.sql</code>	Crea <code>drivers</code> (phone, name, plate, is_available)
002	<code>002_create_trips.sql</code>	Crea <code>trips</code> + índices por status/teléfonos
003	<code>003_create_passengers.sql</code>	Crea <code>passengers</code> ; FK <code>trips.passenger_id</code>
004	<code>004_driver_full_profile.sql</code>	Perfil conductor: cédula, dirección, ciudad, emergencia, vehículo, vencimientos docs + flags de bloqueo
005	<code>005_conversation_sessions.sql</code>	FSM conversacional por teléfono
006	<code>006_document_management.sql</code>	<code>drivers.status</code> , <code>document_reminders</code> , constraints
007	<code>007_conversation_tunnels.sql</code>	<code>conversation_tunnels</code> + <code>tunnel_messages</code>

#	Archivo	Qué hace
008	008_tunnel_closing_status.sql	Status túnel <code>active</code> / <code>closing</code> / <code>closed</code> + índice <code>closes_at</code>
009	009_trip_cancelled_status.sql	Status de trip incluye cancelación
010	010_cancellations_and_policies.sql	Contadores/suspensión conductor + tabla <code>trip_cancellations</code>
011	011_search_and_reassignment.sql	Deadlines de búsqueda / continuar
012	012_trip_driver_exclusions.sql	Exclusiones conductor↔trip
013	013_session_booking_draft.sql	<code>booking_draft</code> JSONB en sessions
014	014_trip_geo_and_fare.sql	Geo pickup/dropoff, distancia, duración, <code>quoted_fare</code> , etc.
015	015_fare_rules.sql	Crea <code>fare_rules</code> (parámetros comerciales)
016	016_fare_increment_80m.sql	<code>increment_meters = 80</code> en reglas activas
017	017_city_context.sql	Tabla <code>cities</code> + <code>city_id</code> en drivers/trips/fare_rules; seed Ibagué
018	018_waiting_flow.sql	Status <code>cancelled_no_driver</code> + contador recordatorios búsqueda
019	019_tariff_final_fare.sql	<code>started_at</code> , <code>finished_at</code> , <code>final_fare</code> , <code>wait_seconds</code>
020	020_fare_rules_tariff_engine.sql	Columnas Tariff Engine en <code>fare_rules</code> (tiempo, espera, etc.)
021	021_sync_ibague_fare_rules_from_seed.sql	Sync montos Ibagué desde seed histórico
022	022_official_ibague_tariffs.sql	Alinea tarifas oficiales de negocio Ibagué
023	023_ibague_night_window_19_to_6.sql	Nocturno 19:00–06:00
024	024_holidays_colombia.sql	Tabla <code>holidays</code> + seed CO 2025–2027
025	025_taximeter_test.sql	<code>taximeter_test_sessions</code> + <code>taximeter_test_runs</code>
026	026_taximeter_test_enrichment.sql	Ruta/polyline/JSON enrichment en taxímetro
027	027_taximeter_test_meter_optional.sql	<code>meter_value</code> / diffs opcionales (MVP sin taxímetro físico)
028	028_ibague_tariff_v2_increment_90.sql	<code>increment_amount = 90</code> (Ibague v2)
029	029_driver_email_operation_card.sql	<code>drivers.email</code> , <code>drivers.operation_expires_at</code>

Archivo auxiliar (no numerado)

Archivo	Notas
supabase/APPLY_SPRINT_18.sql	Script auxiliar histórico de sprint 18; la secuencia canónica son las migraciones numeradas

Cómo aplicar migraciones

1. Abrir SQL Editor del proyecto Supabase (o CLI `supabase db push / psql` si está configurado).
2. Ejecutar cada archivo `001` → `029` en orden, o usar el flujo de migraciones del CLI del equipo.
3. Verificar tablas clave: `drivers`, `trips`, `fare_rules`, `cities`, `holidays`, `taximeter_test_*`.
4. Confirmar fila activa de `fare_rules` para Ibagué y ciudad activa en `cities`.

Datos de configuración esperados post-migración

- Ciudad activa: **Ibagué** (`cities.slug` / flags según 017).
- `fare_rules` activa con parámetros v2 (mínimo, incremento 80 m × \$90, nocturno 19–6, surcharges).
- `holidays` con calendario CO sembrado.

Pendientes de esquema / ops

- Introducir RLS si se expone anon key a clientes.
- Panel admin / vistas BI no existen como migraciones.
- Auth Supabase no requerido para el bot actual.

7. Integración WhatsApp / Meta

Estado de integración

Ítem	Estado
Recepción de mensajes (webhook)	Operativo
Envío texto / botones / location request	Operativo
Verificación GET + firma POST HMAC	Operativo
Notas de voz → Whisper → mismo flujo	Operativo si hay OPENAI_API_KEY
Multi-número / plantillas HSM	No documentado como parte del MVP core
WhatsApp Business App dual-role	Mismo número atiende pasajero y conductor

Endpoint del producto

Método	Path	Rol
GET	/api/webhook	Verificación Meta (hub.mode , hub.verify_token , hub.challenge)
POST	/api/webhook	Eventos entrantes (mensajes)

Implementación: `src/app/api/webhook/route.ts` .

URL de producción típica:

```
https://<dominio-vercel>/api/webhook
```

Configurar esa URL en Meta → App → WhatsApp → Configuration → Webhook.

Tokens y secretos requeridos (nombres)

Variable	Uso
WHATSAPP_TOKEN	Bearer hacia Graph API (enviar mensajes, descargar media)
WHATSAPP_PHONE_NUMBER_ID	ID del número Business
WHATSAPP_VERIFY_TOKEN	Debe coincidir con el verify token configurado en Meta (GET)
WHATSAPP_APP_SECRET	Valida X-Hub-Signature-256 en POST
WHATSAPP_API_VERSION	Versión Graph; default v21.0

Opcionales para voz:

Variable	Uso
OPENAI_API_KEY	Transcripción Whisper

Variable	Uso
<code>OPENAI_WHISPER_MODEL</code>	Default <code>whisper-1</code>
<code>OPENAI_WHISPER_LANGUAGE</code>	Default <code>es</code>
<code>VOICE_TRANSCRIPTION_PROVIDER</code>	Default <code>openai_whisper</code>

Configuración actual (desde código)

1. **GET verify:** si `hub.mode=subscribe` y `token == WHATSAPP_VERIFY_TOKEN` → responde `hub.challenge` (200).
2. **POST:** lee body raw → verifica firma (`src/lib/whatsapp/verify.ts`) → parse (`parse.ts`) → normalize (audio→texto en `normalize-incoming.ts`) → `handleIncomingMessage` .
3. **Envío:** `src/lib/whatsapp/client.ts` usa Graph `/{PHONE_NUMBER_ID}/messages` .
4. **Media:** `src/lib/whatsapp/media.ts` descarga audio con el token.

Flujo de mensajes (alto nivel)

```
Usuario WhatsApp
  → Meta Cloud API
  → POST /api/webhook
  → verify signature
  → parseIncomingMessages
  → normalizeParsedMessage (voz → texto)
  → handleIncomingMessage (Core Agent)
  → módulos (booking / dispatch / drivers / tunnels / ...)
  → sendTextMessage / sendButtonsMessage / location request
  → Meta → Usuario
```

Tipos de interacción soportados

- Texto libre (saludo, intención de viaje, intención conductor, respuestas de formulario).
- Botones interactivos (reply buttons; título ≤ 20 caracteres Meta).
- Ubicación (pin) + location request.
- Audio (nota de voz) si Whisper está configurado.

Mapeo de “superficie” WhatsApp → módulos

Entrada usuario	Módulo
Menú pasajero / “Solicitar servicio” / intención viaje	<code>booking/*</code>
Botones conductor (aceptar, ETA, llegar, iniciar, navegar, finalizar)	<code>dispatch.ts</code>
 /  / “Soy conductor”...	Menú conductor o registro
Registro Continuar/Abandonar/Cancelar/Salir	<code>driver-registration.ts</code>
Taxímetro de prueba (sesión/botones)	<code>taximeter-test/*</code>
Chat durante viaje	<code>tunnels.ts</code>

Entrada usuario	Módulo
Cancelaciones / Ya voy	<code>cancellations.ts</code>
Rating	<code>rating.ts</code>

Pendientes conocidos (Meta / canal)

1. **Plantillas proactivas / fuera de ventana 24h** — no son el foco del MVP actual (conversación reactiva).
2. **Límite de botones** — máximo 3 reply buttons; títulos cortos (p. ej. "Cancelar inscripción" sin emoji 🗑).
3. **Confiabilidad de webhooks** — Meta reintenta; el handler responde 200 incluso si hay error interno (log + swallow) para evitar tormentas; revisar logs Vercel ante fallos silenciosos.
4. **Voz** — sin `OPENAI_API_KEY` las notas de voz no entran al flujo.
5. **Número único** — pasajero y conductor comparten el mismo WABA number; el routing es por rol (existencia en `drivers` + estado de sesión).
6. **Taxímetro vs módulo conductor** — 🚕 / 🚗 bare abren módulo conductor; el taxímetro de prueba se usa vía sesión/botones (no necesariamente el emoji bare). Confirmar UX operativa vigente en código de handler.

Checklist Meta al recuperar entorno

1. App Meta con producto WhatsApp.
2. Número de prueba o producción vinculado.
3. Webhook URL + Verify Token.
4. Suscripción al campo `messages`.
5. Copiar Phone Number ID, Permanent Token (o System User token), App Secret a Vercel.
6. Probar GET challenge y un mensaje "Hola".

8. Conocimiento funcional

Documento de transferencia de conocimiento construido durante el desarrollo del MVP. Mapea conceptos de producto a módulos reales del repositorio.

Visión

WhatXia Mobility es un **MVP de taxi por WhatsApp** (sin app nativa) para **Ibagué**:

- El pasajero solicita, confirma origen/destino y recibe cotización estimada.
- El sistema despacha a conductores disponibles.
- El conductor ejecuta el viaje (ETA → llegada → inicio → navegación → fin).
- La tarifa se estima al inicio y se finaliza al cerrar el viaje (persistencia); al usuario se comunica el **rango estimado** y la regla del taxímetro + recargo de solicitud.
- Conductores se registran y gestionan disponibilidad/documentos por el mismo canal.

Arquitectura del Core Agent

Nombre de producto: Core Agent

Implementación: `src/lib/whatsapp/handler.ts` → `handleIncomingMessage`

Es el orquestador único de entrada. No hay microservicios: un proceso Next.js en Vercel recibe el webhook y enruta por prioridad.

Capas

```
Meta Webhook
→ verify + parse + normalize (voz)
→ Core Agent (handler)
  → tunnels / rating / cancellations / search timeouts
  → dispatch buttons
  → driver menu / registration / update / docs
  → taximeter test
  → booking (Mobility)
  → greeting / intent / fallback
→ WhatsApp client (salida)
→ Supabase (persistencia)
→ Google Maps / OpenAI (servicios externos)
```

Conversation Planner (concepto → código)

No existe un paquete llamado `conversation-planner`. La “planificación” es la **máquina de estados** en:

Persistencia	Código
<code>conversation_sessions.state</code> + drafts	<code>src/lib/sessions.ts</code>
Estados de booking	<code>src/types</code> + <code>src/lib/booking/flow.ts</code>
Estados registro conductor	<code>DRIVER_REGISTERING</code> , <code>DRIVER_REGISTRATION_WELCOME</code> , <code>PAUSED</code> , <code>RESUME_CHOICE</code> , ...

Persistencia	Código
Taxímetro	<code>taximeter_test_sessions.state</code>

El planner efectivo = **orden de if en el handler** + estado en Supabase.

Response Generator (concepto → código)

No hay generador LLM de respuestas de negocio. Las respuestas son **plantillas determinísticas**:

- `sendMessage` / `sendButtonsMessage` (`src/lib/whatsapp/client.ts`)
- Copy embebido en booking, dispatch, registration, rating, etc.

Excepción: **voz entrante** usa Whisper (LLM/ASR) solo para convertir audio→texto; el texto entra al mismo pipeline.

Mobility (Booking)

Código: `src/lib/booking/flow.ts`, `src/lib/booking/intent.ts`

Flujo pasajero (happy path)

1. Saludo / menú o texto con intención de servicio (`parseMobilityIntent`).
2. Origen: pregunta "¿Dónde te recogemos?" → pin WhatsApp y/o texto.
3. Destino: texto → Places (bias ciudad) → confirmación / recuperación.
4. Cotización: `estimateFare` → presentación de **rango** (`present-estimate.ts` , margen +\$3.000).
5. Confirmación → crea `trip` `SEARCHING` → dispatch.

Reglas UX relevantes

- No auto-completar "Punto de recogida" sin interacción.
- Intención libre evita depender siempre de "Hola".
- Mensaje de fin de viaje usa el **mismo quoted_fare** del inicio (rango), no recalcula para mostrar.

Dispatch

Código: `src/lib/dispatch.ts` (+ `search.ts`, `trip-exclusions.ts`, `waiting-flow.ts`)

Ciclo conductor

Oferta → Aceptar/Rechazar → ETA → Ver ubicación / Llegué → Iniciar → Navegar destino → Finalizar.

Comportamientos

- Oferta incluye ubicación nativa WhatsApp + rango estimado.
- Exclusiones tras cancelación/rechazo para reasignar.
- Timeouts de búsqueda + "continuar / cancelar" (waiting flow).
- Al finalizar: persiste `final_fare` vía `finalizeFare`, notifica P/D con rango estimado + copy de taxímetro/\$800, abre rating, programa cierre de túnel.

Pricing / Tariff Engine

Código: `src/lib/tariff/*` (entrada `src/lib/tariff/index.ts`)

Legacy: `src/lib/pricing/*` (deprecado; no SSoT)

SSoT

Fuente	Tabla / artefacto
Parámetros	<code>public.fare_rules</code>
Festivos	<code>public.holidays</code>
Ciudad	<code>public.cities</code> + <code>src/lib/city/context.ts</code>

API

- `estimateFare` — cotización informativa pre-aceptación.
- `finalizeFare` — tarifa oficial al terminar viaje.
- Presentación: `formatEstimatedFareRange*` / `formatCopSymbol`.

Mapa rápido archivo → dominio

Dominio	Path
Core Agent	<code>src/lib/whatsapp/handler.ts</code>
WhatsApp I/O	<code>src/lib/whatsapp/*</code> , <code>src/app/api/webhook</code>
Mobility	<code>src/lib/booking/*</code>
Dispatch	<code>src/lib/dispatch.ts</code>
Search / waiting	<code>src/lib/search.ts</code> , <code>waiting-flow.ts</code>
Tariff	<code>src/lib/tariff/*</code>
Geo	<code>src/lib/geo/*</code>
Drivers	<code>driver-registration</code> , <code>driver-menu</code> , <code>driver-update</code> , <code>supabase/drivers</code>
Tunnels	<code>src/lib/tunnels.ts</code>
Cancellations	<code>src/lib/cancellations.ts</code>
Rating	<code>src/lib/rating.ts</code>
Voice	<code>src/lib/voice/*</code>
Taximeter	<code>src/lib/taximeter-test/*</code>
City	<code>src/lib/city/context.ts</code>

Dominio	Path
Crons	<code>src/app/api/cron/*</code>

9. Reglas de negocio y decisiones de arquitectura

Reglas de negocio tarifarias (Ibagué v2 — resumen)

- Base / piso: tarifa mínima (p. ej. \$6.600).
- Distancia: incrementos sobre excedente tras ~1.600 m; tick 80 m × \$90 (028).
- Recargos configurables: nocturno 19:00–06:00, domingo/festivo, aeropuerto, plataforma.
- Flag `APPLY_CALL_SURCHARGE_ON_ESTIMATE = false`: el recargo de solicitud **no** se suma al estimado mostrado; el copy de fin de viaje recuerda **+\$800** sobre taxímetro.
- UI de estimado: rango `[X, X+3000]` redondeado a centenas.

Decisión de arquitectura

El motor **no** usa `date-holidays` en runtime; solo lee `holidays` en Supabase. Los archivos `city-config/*.ts` son referencia/seed históricos, no fuente operativa.

Registro de conductores

Código: `src/lib/driver-registration.ts`, `src/lib/driver-profile-fields.ts`,
`src/lib/supabase/drivers.ts`

Entrada

Intención conductor ( ,  , "Soy conductor", ...) → si ya existe driver: menú; si no: registro.

Inicio

1. Bienvenida + botones  **Continuar** /  **Abandonar**.
2. Continuar → primera pregunta.
3. Abandonar → sin datos.

Durante el formulario

Botones: **Cancelar inscripción** (borra progreso) /  **Salir** (pausa).

Reentrada: Continuar donde quedó / Empezar de nuevo.

Sin mensajes intermedios "dato guardado".

Campos (orden actual)

Personales: nombre, cédula, email, dirección, ciudad

Vehículo: placa, marca (+ayuda), línea/referencia (+ayuda), color

Documentos: SOAT, técnico-mecánica, tarjeta de operación, licencia de tránsito

Eliminados del flujo: emergencia, año, tipo de servicio.

Cierre

Mensaje de recepción + validación manual del equipo (no auto-activar narrativa de "ya puedes recibir servicios" como única verdad de negocio; el código aún puede crear fila `drivers` según `createDriver` —

ver estado MVP).

Flujo de pasajeros

1. Identidad = teléfono WhatsApp → `passengers` (findOrCreate).
2. Sesión conversacional en `conversation_sessions`.
3. Booking → trip → mensajes de estado → túnel opcional con conductor → fin → rating.
4. Cancelaciones con menú de causales cuando aplica.

Túneles conversacionales

Código: `src/lib/tunnels.ts`

Tablas: `conversation_tunnels`, `tunnel_messages`

Durante el servicio, mensajes no capturados por botones de flujo pueden reenviarse P↔D. Al finalizar: status `closing` + `closes_at`; cron `/api/cron/tunnels` limpia.

Taxímetro de prueba

Código: `src/lib/taximeter-test/*`

Tablas: `taximeter_test_sessions`, `taximeter_test_runs`

Calibración de tarifa en campo **sin** crear trips ni despachar. Independiente de Mobility. MVP simplificado: pins inicio/fin → cálculo WhatXia; `meter_value` opcional.

Nota: el emoji 🚕 bare fue reasignado al **módulo conductor**; no asumir que inicia taxímetro sin revisar handler actual.

Reglas de negocio (lista operativa)

1. Un número WhatsApp puede ser pasajero o conductor según datos/sesión.
2. Solo conductores `available` / no bloqueados reciben ofertas (según lógica dispatch).
3. Documentos vencidos pueden marcar `documents_blocked` / `inactive` (cron docs).
4. Búsqueda tiene deadline; sin conductor → waiting flow.
5. Cancelaciones acumulan políticas / exclusiones.
6. Tarifa mostrada al usuario = **estimada en rango**; cobro de calle = taxímetro + \$800 solicitud (copy).
7. Ciudad operativa actual = Ibagué.
8. Registro conductor puede pausarse; cancelar borra progreso.

Decisiones de arquitectura tomadas

Decisión	Razón
Un solo deploy Next + webhook	Simplicidad MVP, un solo secreto surface

Decisión	Razón
Supabase + service role	Persistencia rápida sin auth de usuario
FSM en DB por teléfono	Conversaciones largas / reanudables
Tariff Engine lee solo DB	Multi-ciudad futura sin redeploy de fórmulas
Holidays en SQL no en npm runtime	Determinismo y ops en un solo lugar
Presentación de rango separada del cálculo	UX sin mutar <code>quoted_fare</code>
Taxímetro sin trips	Calibración sin contaminar operación
Copy determinístico (no LLM de negocio)	Control regulatorio y costos
Voz solo en frontera WhatsApp	Mobility permanece agnóstico al canal de audio
Sin RLS en migraciones	Velocidad MVP; riesgo aceptado con service role

10. Árbol del proyecto

Inventario de rutas del repositorio (sin `node_modules`, `.git` ni `.next`).

Arbol del proyecto WhatXia Mobility MVP

Generado: 2026-07-24

Excluye: node_modules, .git, .next, *.tsbuildinfo

```
.git\  
.next\  
docs\  
node_modules\  
public\  
scripts\  
src\  
supabase\  
.env.example  
.gitignore  
# (.env.local existe solo en máquinas locales; no versionar)  
AGENTS.md  
CLAUDE.md  
eslint.config.mjs  
next-env.d.ts  
next.config.ts  
package-lock.json  
package.json  
README.md  
tsconfig.json  
vercel.json  
docs\backup\  
docs\INFORME-ESTADO-WHATXIA-MOBILITY.md  
docs\backup\01-repositorio.md  
docs\backup\02-vercel.md  
docs\backup\03-supabase.md  
docs\backup\04-whatsapp-meta.md  
docs\backup\05-conocimiento-funcional.md  
docs\backup\06-estado-mvp.md  
docs\backup\07-manual-recuperacion.md  
docs\backup\README.md  
public\file.svg  
public\globe.svg  
public\next.svg  
public\vercel.svg  
public>window.svg  
scripts\diagnose-dispatch-schema.mts  
scripts\generate-co-holidays-sql.ts  
scripts\places-ibague-diag.ts  
scripts\wa-offer-location-probe.ts  
src\app\  
src\lib\  
src\types\  
src\app\api\  
src\app\favicon.ico  
src\app\globals.css  
src\app\layout.tsx  
src\app\page.module.css  
src\app\page.tsx  
src\app\api\cron\  
src\app\api\webhook\  
src\app\api\cron\documents\  
src\app\api\cron\search\  
src\app\api\cron\tunnels\  
src\app\api\cron\documents\route.ts  
src\app\api\cron\search\route.ts  
src\app\api\cron\tunnels\route.ts  
src\app\api\webhook\route.ts  
src\lib\booking\  
src\lib\city\  
src\lib\geo\  

```

```
src\lib\pricing\  
src\lib\supabase\  
src\lib\tariff\  
src\lib\taximeter-test\  
src\lib\voice\  
src\lib\whatsapp\  
src\lib\booking-flow.certify.ts  
src\lib\cancellations.certify.ts  
src\lib\cancellations.ts  
src\lib\city.certify.ts  
src\lib\dispatch.ts  
src\lib\document-jobs.ts  
src\lib\driver-documents.certify.ts  
src\lib\driver-documents.ts  
src\lib\driver-expired-docs-update.ts  
src\lib\driver-intent.certify.ts  
src\lib\driver-menu.ts  
src\lib\driver-profile-fields.ts  
src\lib\driver-registration.ts  
src\lib\driver-update.ts  
src\lib\expired-docs-prompt.ts  
src\lib\intent.certify.ts  
src\lib\places-rank.certify.ts  
src\lib\pricing.certify.ts  
src\lib\rating.ts  
src\lib\routes-map.certify.ts  
src\lib\search.certify.ts  
src\lib\search.ts  
src\lib\sessions.ts  
src\lib\tariff.certify.ts  
src\lib\taximeter-test.certify.ts  
src\lib\trip-exclusions.ts  
src\lib\trips.ts  
src\lib\tunnels.certify.ts  
src\lib\tunnels.ts  
src\lib\waiting-flow.certify.ts  
src\lib\waiting-flow.ts  
src\lib\whatsapp-media.certify.ts  
src\lib\whatsapp-voice.certify.ts  
src\lib\booking\flow.ts  
src\lib\booking\intent.ts  
src\lib\city\context.ts  
src\lib\geo\client.ts  
src\lib\geo\confidence.ts  
src\lib\geo\config.ts  
src\lib\geo\geocoding.ts  
src\lib\geo\maps-url.ts  
src\lib\geo\places.ts  
src\lib\geo\routes.ts  
src\lib\geo\types.ts  
src\lib\pricing\config.ts  
src\lib\pricing\engine.ts  
src\lib\pricing\rules.ts  
src\lib\pricing\surcharges.ts  
src\lib\pricing\types.ts  
src\lib\supabase\client.ts  
src\lib\supabase\drivers.ts  
src\lib\supabase\passengers.ts  
src\lib\tariff\city-config\  
src\lib\tariff\adapters.ts  
src\lib\tariff\calculator.ts  
src\lib\tariff\config-loader.ts  
src\lib\tariff\engine.ts  
src\lib\tariff\holidays.ts  
src\lib\tariff\index.ts  
src\lib\tariff\present-estimate.ts
```

```
src\lib\tariff\provider.ts
src\lib\tariff\types.ts
src\lib\tariff\waiting.ts
src\lib\tariff\city-config\ibague.ts
src\lib\tariff\city-config\medellin.ts
src\lib\tariff\city-config\pasto.ts
src\lib\tariff\city-config\registry.ts
src\lib\taximeter-test\flow.ts
src\lib\taximeter-test\index.ts
src\lib\taximeter-test\store.ts
src\lib\taximeter-test\types.ts
src\lib\voice\openai-whisper.ts
src\lib\voice\provider.ts
src\lib\whatsapp\client.ts
src\lib\whatsapp\handler.ts
src\lib\whatsapp\media.ts
src\lib\whatsapp\normalize-incoming.ts
src\lib\whatsapp\parse.ts
src\lib\whatsapp\types.ts
src\lib\whatsapp\verify.ts
src\types\index.ts
supabase\migrations\
supabase\APPLY_SPRINT_18.sql
supabase\migrations\001_create_drivers.sql
supabase\migrations\002_create_trips.sql
supabase\migrations\003_create_passengers.sql
supabase\migrations\004_driver_full_profile.sql
supabase\migrations\005_conversation_sessions.sql
supabase\migrations\006_document_management.sql
supabase\migrations\007_conversation_tunnels.sql
supabase\migrations\008_tunnel_closing_status.sql
supabase\migrations\009_trip_cancelled_status.sql
supabase\migrations\010_cancellations_and_policies.sql
supabase\migrations\011_search_and_reassignment.sql
supabase\migrations\012_trip_driver_exclusions.sql
supabase\migrations\013_session_booking_draft.sql
supabase\migrations\014_trip_geo_and_fare.sql
supabase\migrations\015_fare_rules.sql
supabase\migrations\016_fare_increment_80m.sql
supabase\migrations\017_city_context.sql
supabase\migrations\018_waiting_flow.sql
supabase\migrations\019_tariff_final_fare.sql
supabase\migrations\020_fare_rules_tariff_engine.sql
supabase\migrations\021_sync_ibague_fare_rules_from_seed.sql
supabase\migrations\022_official_ibague_tariffs.sql
supabase\migrations\023_ibague_night_window_19_to_6.sql
supabase\migrations\024_holidays_colombia.sql
supabase\migrations\025_taximeter_test.sql
supabase\migrations\026_taximeter_test_enrichment.sql
supabase\migrations\027_taximeter_test_meter_optional.sql
supabase\migrations\028_ibague_tariff_v2_increment_90.sql
supabase\migrations\029_driver_email_operation_card.sql
```

11. Manual de recuperación

Objetivo: reconstruir WhatXia Mobility MVP **desde cero** usando solo GitHub, Vercel, Supabase, variables de entorno, migraciones y configuración Meta.

Tiempo estimado (persona familiarizada): 2–4 horas + propagación DNS/Meta.

0. Prerrequisitos

- Cuenta GitHub con acceso a `WhatXia/whatxia-mobility-mvp` (o un fork).
- Cuenta Vercel con permiso para crear proyectos y Cron Jobs.
- Proyecto Supabase (nuevo o vacío).
- App Meta / WhatsApp Cloud API con número de prueba o producción.
- API key Google Maps Platform (Places New, Geocoding, Routes habilitados).
- (Opcional) API key OpenAI para voz.
- Node.js LTS compatible con Next 16 + npm.

1. Clonar el repositorio

```
git clone https://github.com/WhatXia/whatxia-mobility-mvp.git
cd whatxia-mobility-mvp
git checkout main
git pull origin main
npm install
```

Verificar commit de referencia (opcional):

```
git log -1 --oneline
# esperado en el corte del backup: d8a65d7 ...
```

2. Crear / preparar Supabase

1. Crear proyecto en <https://supabase.com>.
2. Copiar:
 - Project URL → `NEXT_PUBLIC_SUPABASE_URL`
 - anon public key → `NEXT_PUBLIC_SUPABASE_ANON_KEY`
 - service_role key → `SUPABASE_SERVICE_ROLE_KEY` (**secreto**)
3. Abrir **SQL Editor**.
4. Aplicar migraciones en orden:

```
supabase/migrations/001_create_drivers.sql
... hasta ...
supabase/migrations/029_driver_email_operation_card.sql
```

Ejecutar **una por una** (o automatizar con Supabase CLI si el equipo lo usa). No saltar números.

5. Verificaciones post-migración:

```
-- Tablas clave
select tablename from pg_tables where schemaname = 'public' order by 1;

-- Ciudad / tarifas
select slug, name from public.cities;
select city_id, active, flag_drop, minimum_fare, increment_meters, increment_amount,
       night_start_hour, night_end_hour
from public.fare_rules where active = true;

-- Festivos
select count(*) from public.holidays where country_code = 'CO';

-- Columnas registro
select column_name from information_schema.columns
where table_name = 'drivers' and column_name in ('email', 'operation_expires_at');
```

- 6. Auth / Storage: **no se requieren** para el bot actual.
- 7. RLS: no hay políticas en migraciones; **no exponer** la service role al cliente.

Detalle de cada migración: [03-supabase.md](#).

3. Variables de entorno (local)

Copiar plantilla:

```
cp .env.example .env.local
```

Completar **valores** (nunca commitear `.env.local`):

Nombre	Obligatorio prod
NEXT_PUBLIC_SUPABASE_URL	Sí
NEXT_PUBLIC_SUPABASE_ANON_KEY	Recomendado
SUPABASE_SERVICE_ROLE_KEY	Sí
WHATSAPP_TOKEN	Sí
WHATSAPP_PHONE_NUMBER_ID	Sí
WHATSAPP_VERIFY_TOKEN	Sí
WHATSAPP_APP_SECRET	Sí
WHATSAPP_API_VERSION	No (default <code>v21.0</code>)
GOOGLE_MAPS_API_KEY	Sí (booking/dispatch/tarifa)
CRON_SECRET	Sí (crons)
OPENAI_API_KEY (+ whisper vars)	Solo si se quiere voz
GEO_CITY_* / PLACE_CONFIDENCE_THRESHOLD	Fallback; SSoT ciudad en DB

Probar local:

```
npm run dev
# Webhook local requiere túnel (ngrok/cloudflared) hacia /api/webhook
```

4. Desplegar en Vercel

1. **Add New Project** → Importar `WhatXia/whatxia-mobility-mvp`.
2. Framework: Next.js (autodetect).
3. Root: `/`.
4. Configurar **Environment Variables** (Production; y Preview si aplica) con los mismos nombres de la tabla anterior.
5. Deploy.
6. Confirmar que `vercel.json` registró crons:
 - `/api/cron/documents` — `0 13 * * *`
 - `/api/cron/tunnels` — `5 13 * * *`
7. Anotar dominio de producción: `https://<proyecto>.vercel.app` o custom domain.
8. Probar build logs: `next build` sin errores.

Detalle: [02-vercel.md](#).

5. Configurar Meta (WhatsApp Cloud API)

1. En Meta Developers → App → WhatsApp → **Configuration**.
2. Webhook callback URL:

```
https://<dominio-produccion>/api/webhook
```

3. Verify token = valor de `WHATSAPP_VERIFY_TOKEN` (exacto).
4. Suscribir campo `messages`.
5. Copiar a Vercel:
 - Temporary/Permanent token → `WHATSAPP_TOKEN`
 - Phone number ID → `WHATSAPP_PHONE_NUMBER_ID`
 - App Secret → `WHATSAPP_APP_SECRET`
6. Redeploy Vercel tras guardar env vars.
7. Enviar mensaje de prueba "Hola" al número Business desde un WhatsApp permitido (modo prueba: números en allowlist).

Detalle: [04-whatsapp-meta.md](#).

6. Google Maps

1. Proyecto GCP con billing.

2. Habilitar: Places API (New), Geocoding API, Routes API.
 3. Restringir la key por IP/referrer según política del equipo (en Vercel suele ser restricción por API + cuotas).
 4. Pegar en `GOOGLE_MAPS_API_KEY`.
-

7. Smoke test de recuperación (checklist)

Pasajero

- ☐ "Hola" → menú solicitar servicio.
- ☐ Flujo origen → destino → cotización con rango.
- ☐ Confirmar → "buscando conductor" (aunque no haya drivers).

Conductor

- ☐ Registrar conductor nuevo (🚗 / intención) → Continuar → completar campos → mensaje final de recepción.
- ☐ Verificar fila en `drivers` (incluye `email`, `operation_expires_at`).
- ☐ Marcar disponible → pedir viaje con otro número → recibir oferta → aceptar → iniciar → finalizar.
- ☐ Verificar mensajes de fin (rango estimado + \$800), no "Tarifa final".

Ops

- ☐ Cron documents responde 401 sin `CRON_SECRET` y 200 con secreto.
- ☐ Logs Vercel sin errores de firma WhatsApp.
- ☐ `fare_rules.increment_amount = 90` para lbagué.

Voz (opcional)

- ☐ Nota de voz → texto → mismo flujo que texto.
-

8. Restaurar solo base de datos (desastre parcial)

Si el código en Vercel está bien pero se perdió la DB:

1. Crear proyecto Supabase nuevo.
2. Reaplicar migraciones 001–029.
3. Actualizar URL/keys en Vercel.
4. Redeploy.
5. Re-sembrar datos operativos (conductores piloto) — **no** hay dump automático en este repo; exportar periódicamente `drivers` / `fare_rules` / `holidays` como backup de datos.

Si se perdió solo `fare_rules` / `holidays`, re-ejecutar migraciones 021–024 y 028 (cuidado: updates idempotentes en su mayoría).

9. Restaurar solo el bot (código)

1. Reimportar repo en Vercel o `git push` a `main`.
2. Reusar mismas env vars.
3. No reaplicar migraciones salvo que el nuevo código exija esquema mayor (hoy: ≤ 029).

10. Lo que este manual NO reconstruye solo

Elemento	Acción humana
Números WhatsApp allowlist / WABA producción	Meta Business Manager
Dominio custom + DNS	Vercel Domains + registrar
Histórico de viajes / corridas taxímetro	Backup SQL externo (no automatizado aquí)
Secretos históricos rotados	Regenerar tokens Meta/Supabase/Google/OpenAI
Aprobaciones de conductores pendientes	Proceso de negocio

11. Orden mental de dependencia

```
GitHub (código)
  → npm install / Vercel build
Supabase (schema 001-029 + keys)
  → env en Vercel
Meta webhook → /api/webhook
Google Maps key
  → booking + tariff + dispatch
(Opcional) OpenAI
  → voz
Smoke test E2E
```

Con esos bloques, cualquier desarrollador puede recuperar el MVP **sin el contexto de chats previos**, usando únicamente este directorio `docs/backup/` y los secretos del equipo.

12. Próximos pasos

Próximos pasos recomendados

1. **Ops:** aplicar migración **029** en todos los entornos; verificar `fare_rules` v2 e `email / operation_expires_at`.
2. **Calibración:** protocolo de corridas con taxímetro de prueba + análisis de sesgo.
3. **Producto registro:** T&C / datos personales; alinear status `drivers` con "pendiente de aprobación".
4. **Hardening:** RLS mínimos o rotación de keys; alertas en Vercel/Supabase.
5. **Cron search** opcional en `vercel.json` si se quiere independencia del tráfico webhook.
6. **Documentación viva:** actualizar este backup tras cada sprint de infraestructura.

Referencias internas

- Conocimiento funcional: [05-conocimiento-funcional.md](#)
- Recuperación: [07-manual-recuperacion.md](#)
- Informe ejecutivo previo: `docs/INFORME-ESTADO-WHATXIA-MOBILITY.md`

Fin del documento

WhatXia Basic — Biblia Técnica MVP v1.0.0

Commit `d8a65d7` · Supabase 29 migraciones · 24 de julio de 2026

Documento generado para archivo y recuperación del MVP.